สร้างระบบ Authentication ด้วย JWT

Lecture 05...entication
PDF

ช่วยอธิบายสไลด์ และสอนผมทำการบ้านหน่อยแบบเป็นขั้นตอน

ยินดีด้วยครับที่คุณมาเรียนรู้เรื่องนี้! สไลด์บรรยายนี้ครอบคลุมคอนเซปต์สำคัญมากๆ ของการทำ Web Security ในยุคปัจจุบันเลยครับ เดี๋ยวผมจะสรุปเนื้อหาในสไลด์ให้ฟังแบบเข้าใจง่ายๆ ก่อน แล้วเราค่อยไปลุยทำการบ้านทีละขั้นตอนกันนะครับ

📖 สรุปเนื้อหาสไลด์: Authentication & Authorization

เนื้อหาหลักของสไลด์นี้คือการทำความเข้าใจว่าเราจะยืนยันตัวตนผู้ใช้งานและจัดการสิทธิ์ของพวกเขาบนเว็บไซต์ได้อย่างไร โดยมีประเด็นสำคัญดังนี้ครับ:

1. ความแตกต่างระหว่าง AuthN และ AuthZ

- **Authentication (AuthN):** คือการตอบคำถามว่า "คุณคือคนที่คุณกล่าวอ้างหรือไม่?" โดยมักจะใช้สิ่งที่รู้ (รหัสผ่าน), สิ่งที่มี (โทรศัพท์มือถือ), หรือสิ่งที่เป็น (ลายนิ้วมือ)
- **Authorization (AuthZ):** คือการตอบคำถามว่า "คุณได้รับอนุญาตให้ทำอะไรได้บ้าง?" เช่น ระบบการจัดการสิทธิ์ตามบทบาท (Role-Based Access Control - RBAC)

2. ปัญหาของ HTTP และการจำสถานะ (State)

- HTTP เป็นโปรโตคอลแบบ Stateless หมายความว่ามันจะไม่จดจำคำขอ (Request) ก่อนหน้าเลย เซิร์ฟเวอร์จึงไม่รู้ว่าคำขอใหม่มาจากผู้ใช้คนเดิมหรือไม่
- **วิธีแก้:** ทุกๆ คำขอจะต้องส่ง "หลักฐานยืนยันตัวตน" ไปด้วยเสมอ

3. วิวัฒนาการของการยืนยันตัวตน

- **Basic Auth:** ส่ง Username และ Password ไปใน Header ของทุกๆ คำขอ (ไม่ปลอดภัยหากข้อมูลรั่วไหลและไม่มีระบบ Log out)
- **Cookies/Sessions:** เซิร์ฟเวอร์เก็บข้อมูลสถานะผู้ใช้ไว้ (Session) และส่ง Cookie (Session ID) ให้เบราว์เซอร์เก็บไว้ส่งกลับมาในครั้งถัดไป
- **Tokens & JWT (JSON Web Tokens):** เซิร์ฟเวอร์ไม่ต้องจำสถานะผู้ใช้อีกต่อไป (Stateless) แต่จะออก Token ที่มีการเซ็นชื่อกำกับให้ Client นำไปเก็บไว้ และส่งมาใน Header (มักจะอยู่ในรูปแบบ `Authorization: Bearer <token>`) ในทุกๆ คำขอ
- **OAuth 2.0 & OIDC:** OAuth 2.0 เป็นกรอบการทำงานเพื่อให้แอปพลิเคชันอื่นเข้าถึงทรัพยากรผู้ใช้ได้โดยไม่ต้องขอรหัสผ่าน ส่วน OIDC (OpenID Connect) สร้างต่อยอดเพื่อใช้ยืนยันตัวตนแบบ Single Sign-On (SSO)
- **SAML 2.0:** ระบบ SSO ระดับองค์กรที่ทำงานด้วยมาตรฐาน XML
- **Passwordless:** การเข้าระบบแบบไม่ใช้รหัสผ่าน เช่น ใช้ลายนิ้วมือ, รหัส OTP, หรือลิงก์ผ่านอีเมล

💻 สอนทำการบ้าน: สร้างระบบ Authentication ด้วย JWT แบบ Step-by-Step

โจทย์การบ้านคือให้สร้าง REST API แบบไม่เก็บสถานะ (Stateless) โดยใช้ Token ในการยืนยันตัวตน และจัดการสิทธิ์ระหว่าง Admin และ User เราจะใช้ Node.js และ Express ในการทำนะครับ

ขั้นตอนที่ 1: เตรียมโปรเจกต์และติดตั้ง Packages

คุณต้องมีแพ็กเกจ `express` สำหรับสร้าง API และ `jsonwebtoken` สำหรับสร้างและตรวจสอบ Token ตามที่โจทย์ระบุ

Bash

```bash
npm init -y
npm install express jsonwebtoken
```

ขั้นตอนที่ 2: สร้างไฟล์หลัก (เช่น `app.js`) และตั้งค่าพื้นฐาน

JavaScript

```javascript
const express = require('express');
const jwt = require('jsonwebtoken');

const app = express();
app.use(express.json()); // สำหรับอ่านข้อมูล JSON จาก Request Body

const SECRET_KEY = "my_super_secret_key"; // ในการทำงานจริงควรเก็บใน Environment Variable
```

ขั้นตอนที่ 3: สร้าง Route `POST /login` สำหรับออก Token

โจทย์บอกว่าให้รับค่า `username` ถ้าเป็น "admin" ให้ใส่ payload `{ role: 'admin' }` ถ้าไม่ใช่ให้เป็น `{ role: 'user' }` แล้วคืนค่า Token กลับไป

JavaScript

```javascript
app.post('/login', (req, res) => {
    const { username } = req.body; // รับค่า username

    let payload = {};
    if (username === 'admin') {
        payload = { role: 'admin' }; // [cite: 119]
    } else {
        payload = { role: 'user' }; // [cite: 120]
    }

    // สร้าง Token (Sign JWT) [cite: 120]
    const token = jwt.sign(payload, SECRET_KEY, { expiresIn: '1h' });

    res.json({ token: token }); // คืนค่า token ให้ Client [cite: 120]
});
```

ขั้นตอนที่ 4: สร้าง Authentication Middleware (ยืนยันตัวตน)

โจทย์ให้สร้าง Middleware สำหรับดึง Token จาก `Authorization: Bearer <token>` ตรวจสอบความถูกต้อง และนำ payload ไปแปะไว้ที่ `req.user`

JavaScript

```javascript
const authenticateToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    // ดึงส่วน <token> ออกมาจากตัวอักษร "Bearer <token>" [cite: 121]
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) return res.status(401).json({ message: "Access Denied" });

    // ตรวจสอบ (Verify) Token [cite: 121]
    jwt.verify(token, SECRET_KEY, (err, decodedPayload) => {
        if (err) return res.status(403).json({ message: "Invalid Token" });

        // แนบ payload ไปกับ req.user [cite: 122]
        req.user = decodedPayload;
        next(); // ให้ไปทำงานในฟังก์ชันถัดไป
    });
};
```

ขั้นตอนที่ 5: สร้าง Authorization Middleware (จัดการสิทธิ์ Admin)

โจทย์ให้สร้าง Middleware `checkAdmin` เช็คว่า `req.user.role` เป็น 'admin' หรือไม่ ถ้าไม่ใช่ให้ตอบกลับด้วย Status 403

JavaScript

```javascript
const checkAdmin = (req, res, next) => {
    // เช็คว่า role คือ admin หรือไม่ [cite: 122]
    if (req.user && req.user.role === 'admin') {
        next(); // อนุญาตให้ผ่านได้
    } else {
        res.status(403).json({ message: "Forbidden: Admins Only" }); // คืนค่า 403
    }
};
```

ขั้นตอนที่ 6: สร้าง Protected Route `/admin-only`

นำ Middleware ทั้งสองตัวมาประกอบร่างเข้ากับ Route ที่ต้องการปกป้อง

ขออธิบายการบ้านละเอียดหน่อยพร้อมโครงสร้างการเขียนแบบเป็นขั้นตอน

JavaScript

```javascript
// ใช้ middlewares ตามลำดับ: ยืนยันตัวตนก่อน -> แล้วค่อยตรวจสอบสิทธิ์
app.get('/admin-only', authenticateToken, checkAdmin, (req, res) => {
    res.json({ message: "Welcome Admin! You have access to this route." });
});

app.listen(3000, () => {
    console.log('Server is running on port 3000');
});
```

เข้าใจเลยครับว่าโจทย์การบ้านนี้อาจจะดูมีหลายส่วนประกอบกัน แต่จริงๆ แล้วพี่อเราแบ่งเป็นขั้นตอนตามโครงสร้างของ Express.js มันจะทำความเข้าใจได้ง่ายขึ้นเยอะครับ เรากำลังจะสร้าง REST API สำหรับแอปพลิเคชันบนมือถือที่เซิร์ฟเวอร์จะไม่เก็บสถานะ (Session State) ของผู้ใช้เอาไว้เลย แต่จะใช้วิธีออก Token ให้ฝั่ง Client ส่งแนบมากับทุกๆ คำขอแทน

เดี๋ยวเราไปเริ่มกันที่ตัวเซิร์ฟเวอร์ สร้างการเขียนโค้ดไปทีละขั้นเลยนะครับ

โครงสร้างและขั้นตอนการทำการบ้าน

ขั้นตอนที่ 1: ติดตั้ง Packages ที่จำเป็น

โจทย์ข้อนี้ต้องใช้โค้ดสำหรับเข้ารหัสและถอดรหัส Token ซึ่งเราไม่ควรเขียนเองแต่ควรใช้ Library ที่มีมาตรฐานรองรับ โดยเราจะใช้ `jsonwebtoken` ส่วนตัวเซิร์ฟเวอร์เราจะใช้ `express` เป็นโครงสร้างพื้นฐานในการทำ REST API ครับ เพื่อยิง Request ทดสอบดูได้เลย

Bash

```bash
npm install express jsonwebtoken
```

ขั้นตอนที่ 2: สร้าง Route สำหรับ Login (`POST /login`)

ส่วนนี้ทำหน้าที่เปรียบเสมือนด่านแรกในการรับข้อมูลผู้ใช้และออกบัตรผ่าน (Token)

- **สิ่งที่ต้องทำ:** สร้าง Route ที่รับค่า `username` เข้ามา
- **เงื่อนไข:** * ถ้า `username` คือ "admin" ให้สร้าง Token ที่มี Payload เป็น `{ role: 'admin' }`
    - ถ้าเป็นชื่ออื่นๆ ให้สร้าง Token ที่มี Payload เป็น `{ role: 'user' }`

- **ผลลัพธ์:** ส่ง Token นั้นกลับไปให้ Client

JavaScript

```javascript
const express = require('express');
const jwt = require('jsonwebtoken');
const app = express();
app.use(express.json());

const SECRET_KEY = 'my_secret_key';

// สร้าง POST /login route
app.post('/login', (req, res) => {
    const { username } = req.body; // รับค่า username

    // ตรวจสอบเงื่อนไขเพื่อกำหนด payload
    const payload = username === 'admin' ? { role: 'admin' } : { role: 'user' };

    // สร้าง Token
    const token = jwt.sign(payload, SECRET_KEY);

    // คืนค่า Token ให้ Client
    res.json({ token });
});
```

ขั้นตอนที่ 3: สร้าง Authentication Middleware (ตัวตรวจสอบ Token)

Middleware ตัวนี้จะทำงานทุกครั้งที่มีการเรียกใช้งาน Route ที่เราต้องการปกป้อง เพื่อดูว่าผู้ใช้มีบัตรผ่านที่ถูกต้องไหม

- **สิ่งที่ต้องทำ:** ดึงค่า Token ออกมาจาก Header ที่ชื่อว่า `Authorization: Bearer <token>`
- **การตรวจสอบ:** ทำการ Verify Token นั้น
- **ผลลัพธ์:** ถ้า Token ถูกต้อง ให้เอา Payload ที่ถอดรหัสได้ไปแนบไว้กับ `req.user`

JavaScript

```javascript
const authenticateToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1]; // ดึง <token> ออกจาก 'Bearer <token>'

    if (!token) return res.sendStatus(401);

    // Verify token
    jwt.verify(token, SECRET_KEY, (err, user) => {
        if (err) return res.sendStatus(403);

        // แนบ payload ไปกับ req.user
        req.user = user;
        next(); // ส่งไม้ต่อให้ฟังก์ชันถัดไป
    });
};
```

ขั้นตอนที่ 4: สร้าง Authorization Middleware (`checkAdmin`)

Middleware ตัวนี้จะทำงานต่อจากตัวด่านบน เพื่อเช็คว่าถึงแม้จะมีบัตรผ่าน แต่มี "สิทธิ์" ระดับผู้ดูแลระบบหรือไม่

- **สิ่งที่ต้องทำ:** ตรวจสอบค่า `req.user.role` ว่าเป็น `'admin'` แบบเป๊ะๆ หรือไม่
- **ผลลัพธ์:** ถ้าไม่ใช่ 'admin' ให้ตอบกลับด้วย Status `403 Forbidden`

JavaScript

```javascript
const checkAdmin = (req, res, next) => {
    // เช็คว่า role คือ admin หรือไม่
    if (req.user.role !== 'admin') {
        return res.status(403).json({ message: 'Forbidden' }); // คืนค่า 403 Forbidden
    }
```

```
    next();
};
```

ขั้นตอนที่ 5: สร้าง Protected Route (`/admin-only`)
นำทุกอย่างมาประกอบกันเพื่อสร้างเส้นทางที่สงวนไว้สำหรับ Admin เท่านั้น

- **สิ่งที่ต้องทำ:** สร้าง Route `/admin-only` โดยให้เรียกใช้ Middleware ทั้งสองตัวที่เราเพิ่งสร้างขึ้นมา

JavaScript

```javascript
// นำ middleware ทั้งสองตัวมาใช้งาน
app.get('/admin-only', authenticateToken, checkAdmin, (req, res) => {
    res.json({ message: 'Welcome to the admin dashboard!' });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

เมื่อคุณนำโค้ดในแต่ละขั้นตอนมารวมกันไว้ในไฟล์เดียว (เช่น `index.js`) คุณก็จะได้แอปพลิเคชันที่ตรงตามโจทย์เป๊ะเลยครับ

อยากให้ผมแนะนำวิธีทดสอบ API ตัวนี้ (เช่น การส่ง Request เข้าไปเพื่อดูการทำงานจริง) ด้วยเครื่องมืออย่าง Postman หรือ cURL ต่อเลยไหมครับ?

ได้เลย

จัดไปครับ! การทดสอบ API เป็นขั้นตอนที่สำคัญมาก เพื่อให้ชัวร์ว่าโค้ดที่เราเขียนทำงานได้ถูกต้องตามโจทย์เป๊ะๆ โดยเราจะจำลองว่าเราเป็นแอปพลิเคชันบนมือถือที่ส่ง Request ไปหาเซิร์ฟเวอร์ครับ

ก่อนอื่น **อย่าลืมเปิดรันเซิร์ฟเวอร์ของคุณทิ้งไว้** นะครับ (เปิด Terminal แล้วพิมพ์ `node ชื่อไฟล์ของคุณ.js` เช่น `node index.js`)

ผมจะสอนวิธีทดสอบ 2 แบบนะครับ เลือกใช้แบบที่คุณถนัดได้เลย:

วิธีที่ 1: ทดสอบด้วย Postman (แนะนำ เพราะมีหน้าตา UI ใช้งานง่าย)

Step 1: ทดสอบเข้าสู่ระบบ (Login) เพื่อรับ Token

1. เปิด Postman ขึ้นมา สร้าง Request ใหม่
2. เปลี่ยน HTTP Method จาก `GET` เป็น `POST`
3. ใส่ URL: `http://localhost:3000/login`
4. ไปที่แถบ **Body** ➡️ เลือก **raw** ➡️ เปลี่ยนข้อความด้านขวาสุดจาก Text เป็น **JSON**
5. ใส่ข้อมูลจำลองว่าเป็น Admin:

   JSON

   ```json
   {
       "username": "admin"
   }
   ```

6. กดปุ่ม **Send**
7. **ผลลัพธ์ที่ได้:** คุณจะเห็นกล่องข้อความตอบกลับมาเป็น JSON พร้อมกับสายอักขระยาวๆ (นั่นคือ Token) ให้ **Copy Token นั้นเก็บไว้ครับ** *(ลองเปลี่ยน username เป็นชื่ออื่น เช่น "johndoe" แล้วกด Send อีกรอบ เพื่อ Copy Token ของ User ธรรมดาเก็บไว้อีกอันเพื่อเปรียบเทียบด้วยนะครับ)*

Step 2: ทดสอบการเข้าถึง Route ที่ป้องกันไว้ (`/admin-only`)

1. สร้าง Request ใหม่อีกอันใน Postman
2. เลือก HTTP Method เป็น `GET`
3. ใส่ URL: `http://localhost:3000/admin-only`
4. ไปที่แถบ **Authorization** ➡️ ตรงช่อง Type ให้เลือก **Bearer Token**
5. นำ Token ที่ Copy ไว้จาก Step 1 มาวางในช่อง Token ทางด้านขวา
6. กดปุ่ม **Send**

ลองทดสอบ 3 กรณีนี้ดูครับ:

- **กรณีที่ 1 (ใส่ Token ของ Admin):** คุณควรจะได้ Status `200 OK` และข้อความต้อนรับ
- **กรณีที่ 2 (เปลี่ยนไปใส่ Token ของ User ธรรมดา):** คุณควรจะได้ Status `403 Forbidden` เพราะ Middleware `checkAdmin` ทำงานและบล็อกไว้
- **กรณีที่ 3 (ไม่ใส่ Token เลย / เลือก Type เป็น No Auth):** คุณควรจะได้ Status `401 Unauthorized` เพราะ Middleware ตัวแรกจับได้ว่าไม่มีบัตรผ่าน

วิธีที่ 2: ทดสอบด้วย cURL (ผ่าน Terminal / Command Prompt)

ถ้าคุณสายพิมพ์คำสั่ง สามารถก๊อปปี้คำสั่งเหล่านี้ไปรันใน Terminal ที่ว่างๆ อีกหน้าต่างนึงได้เลยครับ (บน Windows ถ้าใช้ Command Prompt อาจจะต้องเปลี่ยนฟันหนูเล็กน้อย แนะนำให้รันบน PowerShell หรือ Git Bash ครับ)

1. ขอ Token สำหรับ Admin:

```bash
curl -X POST http://localhost:3000/login \
-H "Content-Type: application/json" \
-d '{"username": "admin"}'
```

(Copy Token ที่ได้จากหน้าจอไว้)

2. ขอ Token สำหรับ User ธรรมดา:

```bash
curl -X POST http://localhost:3000/login \
-H "Content-Type: application/json" \
-d '{"username": "johndoe"}'
```

(Copy Token ของคนนี้เก็บไว้อีกอัน)

3. ทดสอบยิงไปที่ /admin-only:
นำ Token ที่ Copy ไว้มาแทนที่ตรงคำว่า `ใส่_TOKEN_ตรงนี้` (ลบวงเล็บออกด้วยนะครับ)

```bash
curl -X GET http://localhost:3000/admin-only \
-H "Authorization: Bearer ใส่_TOKEN_ตรงนี้"
```

ลองเอา Token ทั้งสองแบบมาสลับเทสดูได้เลยครับ!

ถ้าทดสอบแล้วติด Error ตรงไหน หรือได้ผลลัพธ์ไม่ตรงตามที่คาดหวัง ก๊อปปี้ข้อความ Error มาให้ผมช่วยดูได้เลยนะครับ ทำได้ไหมเอ่ย?

เปิด postman ยังไง

การเปิดใช้งาน Postman ไม่ยากเลยครับ! แต่ก่อนอื่นต้องเช็คก่อนว่าในเครื่องของคุณมีโปรแกรมนี้ติดตั้งไว้หรือยัง

ลองทำตามขั้นตอนนี้ได้เลยครับ:

กรณีที่ 1: ยังไม่มีโปรแกรม Postman ในเครื่อง

ถ้าคุณยังไม่เคยโหลดมาเลย ให้ทำตามนี้ครับ:

1. **ดาวน์โหลด:** เข้าไปที่เว็บไซต์ https://www.postman.com/downloads/ แล้วกดปุ่มดาวน์โหลดให้ตรงกับระบบปฏิบัติการของคุณ (Windows หรือ Mac)
2. **ติดตั้ง:** ดับเบิลคลิกไฟล์ที่โหลดมา แล้วกด Install ตามขั้นตอนปกติเหมือนลงโปรแกรมทั่วไป
3. **เข้าสู่ระบบ (ไม่บังคับ):** เมื่อเปิดโปรแกรมขึ้นมาครั้งแรก มันอาจจะให้คุณสมัครสมาชิกหรือ Login (สามารถใช้ Google Account ล็อกอินได้เลยเพื่อความสะดวก หรือจะกดข้าม/Skip ไปหน้าใช้งานเลยก็ได้ครับ)

กรณีที่ 2: มีโปรแกรมในเครื่องแล้ว

1. **สำหรับ Windows:** กดปุ่ม Start (โลโก้ Windows) บนคีย์บอร์ด พิมพ์คำว่า `Postman` แล้วกด Enter หรือคลิกที่ไอคอนโปรแกรมสีส้มๆ รูปคนบินได้ครับ
2. **สำหรับ Mac:** กด `Command + Space` เพื่อเปิด Spotlight Search พิมพ์ `Postman` แล้วกด Enter

---

เปิดขึ้นมาแล้ว ทำยังไงต่อ? (เริ่มสร้าง Request)

พอโปรแกรมเปิดขึ้นมาปุ๊บ หน้าตาอาจจะดูมีปุ่มเยอะแยะไปหมด ไม่ต้องตกใจครับ ให้โฟกัสแค่การสร้าง **"HTTP Request"** ใหม่ ตามนี้เลย:

1. มองหาปุ่ม **เครื่องหมายบวก (+)** หรือปุ่ม **"New"** (มักจะอยู่มุมซ้ายบน หรือตรงกลางหน้าจอ)
2. เลือก **"HTTP Request"** 3. คุณจะได้หน้าต่างใหม่ที่หน้าตาคล้ายๆ แถบใส่ URL ของเว็บเบราว์เซอร์
3. ตรงนี้แหละครับที่คุณสามารถเปลี่ยน Method (จาก `GET` เป็น `POST`) และกรอก URL `http://localhost:3000/login` ตามที่เราคุยกันไว้ในข้อความที่แล้วได้เลย!

💡 **ทริคเสริม:** ถ้าคุณไม่อยากลงโปรแกรมในเครื่อง Postman เขามีเวอร์ชันบนเว็บด้วยนะครับ (ใช้ผ่านเบราว์เซอร์ได้เลยที่เว็บของเขา) แต่เวอร์ชันโปรแกรมในเครื่องจะทำงานได้เสถียรกว่าสำหรับการเทสแบบ Localhost (เซิร์ฟเวอร์ในเครื่องเราเอง) ครับ

เปิดโปรแกรมขึ้นมาได้เรียบร้อยไหมครับ? ถ้าเปิดได้แล้ว ลองทำตาม Step 1 (ทดสอบยิง `POST /login`) ในข้อความก่อนหน้าดูได้เลย ติดตรงไหนบอกผมนะ!

ใช้ swagger สิ

ไอเดียเยี่ยมมากครับ! ใช้ Swagger ถือว่าโปรสุดๆ เพราะเราไม่ต้องง้อโปรแกรมอื่นเลย สามารถกดเทส API ผ่านหน้าเว็บเบราว์เซอร์ได้ทันที แถมยังได้ Document สวยๆ เอาไว้อ่านด้วย

การจะใช้ Swagger ใน Express เราจะต้องเขียนโค้ดเพิ่มอีกนิดหน่อยครับ โดยการใส่คอมเมนต์ (JSDoc) ไว้บน Route เพื่อให้ Swagger รู้ว่าต้องสร้างหน้าตายังไง

เดี๋ยวเรามาทำตามขั้นตอนนี้กันเลยครับ:

ขั้นตอนที่ 1: ติดตั้งแพ็กเกจ Swagger

เราต้องลงตัวอ่านและตัวแสดงผล Swagger เพิ่มครับ เปิด Terminal แล้วพิมพ์คำสั่งนี้:

```
Bash
```

```
npm install swagger-ui-express swagger-jsdoc
```

ขั้นตอนที่ 2: อัปเดตโค้ดในไฟล์ของคุณ

ผมเอาโค้ดการบ้านเดิม มาเพิ่มส่วนตั้งค่า Swagger และใส่คอมเมนต์เพื่อให้มันสร้างหน้าจอ UI ให้ครับ (สมมติว่าไฟล์นี้ชื่อ `index.js`)

JavaScript

```javascript
const express = require('express');
const jwt = require('jsonwebtoken');
const swaggerUi = require('swagger-ui-express');
const swaggerJsDoc = require('swagger-jsdoc');

const app = express();
app.use(express.json());

const SECRET_KEY = 'my_secret_key';

// --- 1. ตั้งค่า Swagger ---
const swaggerOptions = {
  swaggerDefinition: {
    openapi: '3.0.0',
    info: {
      title: 'Homework Auth API',
      version: '1.0.0',
      description: 'ระบบ Authentication ด้วย JWT',
    },
    components: {
      securitySchemes: {
        bearerAuth: {
          type: 'http',
          scheme: 'bearer',
          bearerFormat: 'JWT',
        },
      },
    },
  },
  // ใส่ชื่อไฟล์ที่คุณใช้รันโค้ดนี้ (เช่น index.js)
  apis: ['index.js'],
};

const swaggerDocs = swaggerJsDoc(swaggerOptions);
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocs));

// --- 2. Middlewares ---
const authenticateToken = (req, res, next) => {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) return res.status(401).json({ message: 'Unauthorized: ไม่พบ Token' });

    jwt.verify(token, SECRET_KEY, (err, user) => {
        if (err) return res.status(403).json({ message: 'Forbidden: Token ไม่ถูกต้อง' });
        req.user = user;
        next();
    });
};

const checkAdmin = (req, res, next) => {
    if (req.user.role !== 'admin') {
        return res.status(403).json({ message: 'Forbidden: สำหรับ Admin เท่านั้น' });
    }
    next();
};

// --- 3. Routes ---
```

```
/**
 * @swagger
 * /login:
 * post:
 * summary: ส่ง Username เพื่อรับ Token
 * requestBody:
 * required: true
 * content:
 * application/json:
 * schema:
 * type: object
 * properties:
 * username:
 * type: string
 * example: "admin"
 * responses:
 * 200:
 * description: เข้าสู่ระบบสำเร็จ ได้รับ Token
 */
app.post('/login', (req, res) => {
    const { username } = req.body;
    const payload = username === 'admin' ? { role: 'admin' } : { role: 'user' };
    const token = jwt.sign(payload, SECRET_KEY, { expiresIn: '1h' });
    res.json({ token });
});

/**
 * @swagger
 * /admin-only:
 * get:
 * summary: เข้าถึงหน้าแดชบอร์ดของ Admin
 * security:
 * - bearerAuth: []
 * responses:
 * 200:
 * description: ยืนยันตัวตนผ่านและเป็น Admin
 * 401:
 * description: ไม่ได้ส่ง Token
 * 403:
 * description: ส่ง Token แต่ไม่ใช่ Admin
 */
app.get('/admin-only', authenticateToken, checkAdmin, (req, res) => {
    res.json({ message: 'Welcome Admin! You have access to this route.' });
});

// --- 4. เริ่มเซิร์ฟเวอร์ ---
app.listen(3000, () => {
    console.log('Server is running on port 3000');
    console.log('👉 เปิด Swagger ได้ที่ http://localhost:3000/api-docs');
});
```

ขั้นตอนที่ 3: วิธีทดสอบผ่านหน้าเว็บ

1. รันเซิร์ฟเวอร์ด้วยคำสั่ง `node index.js`
2. เปิดเว็บเบราว์เซอร์ไปที่ **http://localhost:3000/api-docs**
3. **วิธี Login:** กดที่แถบ `POST /login` -> กดปุ่ม **Try it out** มุมขวาบน -> แก้ไขช่อง `username` เป็น "admin" -> กดปุ่ม **Execute**
4. เลื่อนลงมาดูตรง Server response ให้ Copy ตัวอักษร Token ที่ได้มา
5. **วิธียืนยันตัวตน:** เลื่อนขึ้นไปด้านบนสุด กดปุ่ม 🟢 **Authorize** ที่เป็นรูปแม่กุญแจ -> นำ Token ที่ Copy ไว้ไปวางในช่อง -> กด Authorize
6. **วิธีเข้าหน้า Admin:** กดที่แถบ `GET /admin-only` -> กด **Try it out** -> กด **Execute** (ถ้าถูกต้อง จะได้ Status 200 พร้อมข้อความต้อนรับครับ)

ลองรันดูเลยครับ! ติดขัดตรงไหน หรือกดรันแล้วมี Error แจ้งผมได้ทันทีเลยนะครับ

ไม่ใช้ swaggerjsdoc ให้สร้าง swagger.yaml

เข้าใจแล้วครับ! การแยกไฟล์เอกสารออกมาเป็น `swagger.yaml` เป็นวิธีปฏิบัติที่ดีมาก (Best Practice) เพราะจะช่วยให้ไฟล์โค้ด JavaScript ของเราสะอาดและอ่านง่ายขึ้นเยอะเลยครับ ไม่ต้องมีคอมเมนต์ยาวๆ มารกหูรกตา

ถ้าอย่างนั้นเรามาปรับวิธีทำกันใหม่ โดยเราจะใช้แพ็กเกจ `yamljs` เพื่อมาช่วยอ่านไฟล์ YAML แทน `swagger-jsdoc` นะครับ

ขั้นตอนที่ 1: ติดตั้งแพ็กเกจที่จำเป็น

ลบ `swagger-jsdoc` ออก (ถ้าลงไปแล้ว) แล้วติดตั้ง `yamljs` เพิ่มแทนครับ

Bash

```bash
npm install express jsonwebtoken swagger-ui-express yamljs
```

ขั้นตอนที่ 2: สร้างไฟล์ `swagger.yaml`

สร้างไฟล์ใหม่ชื่อ `swagger.yaml` ไว้ในโฟลเดอร์เดียวกันกับโปรเจกต์ของคุณ แล้วก๊อปปี้โค้ดด้านล่างนี้ไปใส่ได้เลยครับ (ระวังเรื่องการย่อหน้าเว้นวรรคด้วยนะครับ เพราะ YAML อ่อนไหวเรื่องนี้มาก)

YAML

```yaml
openapi: 3.0.0
info:
  title: Homework Auth API
  version: 1.0.0
  description: ระบบ Authentication ด้วย JWT ตามโจทย์การบ้าน
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
paths:
  /login:
    post:
      summary: ส่ง Username เพื่อรับ Token
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                username:
                  type: string
                  example: "admin"
      responses:
        '200':
          description: เข้าสู่ระบบสำเร็จ ได้รับ Token
          content:
            application/json:
              schema:
                type: object
                properties:
                  token:
                    type: string
  /admin-only:
    get:
```

```
      summary: เข้าถึงหน้าแดชบอร์ดของ Admin
      security:
        - bearerAuth: []
      responses:
        '200':
          description: ยืนยันตัวตนผ่านและเป็น Admin
        '401':
          description: ไม่ได้ส่ง Token ใน Header
        '403':
          description: ส่ง Token แต่ไม่ใช่ Admin หรือ Token ไม่ถูกต้อง
```

ขั้นตอนที่ 3: ปรับโค้ดในไฟล์ `index.js`

ทีนี้เราก็ดึงไฟล์ `swagger.yaml` มาใช้งานใน Express แบบคลีนๆ ได้เลยครับ ตามข้อกำหนดของการบ้านที่ให้สร้าง REST API แบบไม่เก็บสถานะ (Stateless) และจัดการสิทธิ์ (Admin vs. User):

JavaScript

```javascript
const express = require('express');
const jwt = require('jsonwebtoken'); // ใช้ jsonwebtoken package [cite: 514]
const swaggerUi = require('swagger-ui-express');
const YAML = require('yamljs');

const app = express();
app.use(express.json());

const SECRET_KEY = 'my_secret_key';

// 1. โหลดไฟล์ swagger.yaml มาใช้งาน
const swaggerDocument = YAML.load('./swagger.yaml');
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));

// 2. Route /login สำหรับขอ Token [cite: 515]
app.post('/login', (req, res) => {
    const { username } = req.body; // Accepts username [cite: 516]

    // ถ้า username คือ "admin" ให้ sign token ด้วย payload { role: 'admin' } [cite: 517]
    // ถ้าไม่ใช่ ให้ sign token ด้วย payload { role: 'user' } [cite: 518]
    const payload = username === 'admin' ? { role: 'admin' } : { role: 'user' };

    const token = jwt.sign(payload, SECRET_KEY, { expiresIn: '1h' });
    res.json({ token }); // Return the token to the client [cite: 519]
});

// 3. Authentication Middleware [cite: 520]
const authenticateToken = (req, res, next) => {
    // Extracts the token from the Authorization: Bearer <token> header [cite: 521]
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) return res.status(401).json({ message: 'Unauthorized: ไม่พบ Token' });

    // Verifies the token. If valid, attach the payload to req.user [cite: 522]
    jwt.verify(token, SECRET_KEY, (err, user) => {
        if (err) return res.status(403).json({ message: 'Forbidden: Token ไม่ถูกต้อง' });
        req.user = user;
        next();
    });
};

// 4. Authorization Middleware (checkAdmin) [cite: 523]
const checkAdmin = (req, res, next) => {
    // Checks if req.user.role is exactly 'admin' [cite: 524]
    if (req.user.role !== 'admin') {
        // If not, return status 403 Forbidden [cite: 525]
        return res.status(403).json({ message: 'Forbidden: สำหรับ Admin เท่านั้น' });
    }
```

```
    next();
};

// 5. Protected route /admin-only that uses both middlewares [cite: 526]
app.get('/admin-only', authenticateToken, checkAdmin, (req, res) => {
    res.json({ message: 'Welcome Admin! You have access to this route.' });
});

// เริ่มเซิร์ฟเวอร์
app.listen(3000, () => {
    console.log('Server is running on port 3000');
    console.log('👉 เปิดหน้า Swagger UI ได้ที่ http://localhost:3000/api-docs');
});
```

วิธีนี้โค้ดจะดูเป็นระเบียบและจัดการง่ายขึ้นมากครับ คุณสามารถลองรัน `node index.js` แล้วเข้าไปทดสอบผ่านหน้าเว็บ `http://localhost:3000/api-docs` ตามวิธีเดิมได้เลย

คุณอยากให้ผมแนะนำวิธีการเพิ่ม Route อื่นๆ ลงในไฟล์ `swagger.yaml` เพิ่มเติมไหมครับ?

ทดสอบยังไง

การทดสอบผ่าน Swagger UI ที่เราเพิ่งสร้างขึ้นมานั้นง่ายและเห็นภาพชัดเจนมากครับ ทำตามขั้นตอนนี้ได้เลย:

ขั้นที่ 1: รันเซิร์ฟเวอร์
เปิด Terminal ขึ้นมาแล้วพิมพ์คำสั่งรันไฟล์ของคุณ (สมมติว่าชื่อ `index.js`)

Bash

```bash
node index.js
```

ขั้นที่ 2: เปิดหน้าต่างทดสอบ (Swagger UI)
เปิดเว็บเบราว์เซอร์ (เช่น Chrome หรือ Edge) แล้วเข้าไปที่:
👉 `http://localhost:3000/api-docs`

ขั้นที่ 3: ทดสอบเข้าสู่ระบบเพื่อเอา Token (POST /login)

1. คลิกที่แถบสีเขียว `POST /login` เพื่อกางเมนูออก
2. คลิกปุ่ม **"Try it out"** (มุมขวาบนของกล่อง)
3. ในกล่อง Request body จะมีข้อมูลตัวอย่าง `{"username": "admin"}` โผล่มา
4. กดปุ่มแถบสีฟ้า **"Execute"** 5. เลื่อนลงมาดูตรง **Server response** คุณจะเห็น `Code 200` และในช่อง Response body จะมี Token ยาวๆ โผล่มา ให้ **Copy ตัวหนังสือยาวๆ นั้นไว้ครับ** (ก็อปแค่ตัว token ไม่เอาปีกกาและเครื่องหมายคำพูด)

ขั้นที่ 4: ใส่รหัสผ่านประจำตัว (Authorize)

1. เลื่อนกลับไปด้านบนสุดของหน้าเพจ มองหาปุ่ม 🟢 **"Authorize"** (มีรูปกุญแจ) แล้วคลิกเลย
2. จะมีหน้าต่างเด้งขึ้นมา ให้นำ Token ที่ Copy ไว้เมื่อกี้ไปวางในช่อง **"Value"**
3. กดปุ่ม **"Authorize"** แล้วกด **"Close"** เพื่อปิดหน้าต่าง (ตอนนี้ Swagger จำ Token ของคุณไว้แล้ว)

ขั้นที่ 5: ทดสอบเข้าใช้งานเส้นทางของ Admin (GET /admin-only)

1. คลิกที่แถบสีฟ้า `GET /admin-only`
2. กดปุ่ม **"Try it out"**
3. กดปุ่ม **"Execute"**
4. เลื่อนลงมาดูผลลัพธ์: ถ้าทุกอย่างถูกต้อง คุณจะได้ `Code 200` และข้อความ `"Welcome Admin! You have access to this route."`

🔥 **ลองทดสอบกรณีอื่นๆ (เพื่อให้ชัวร์ว่าระบบป้องกันทำงานจริง):**

- **ทดสอบว่าเป็น User ธรรมดาเข้าได้ไหม:** กลับไปที่ `POST /login` เปลี่ยน username เป็นชื่ออื่น (เช่น `"user"`) -> กด Execute -> Copy Token อันใหม่มา -> ไปกดปุ่ม Authorize ด้านบนสุดเพื่อ Logout ของเก่าแล้วใส่ Token อันใหม่เข้าไป -> ลองยิง `GET /admin-only` อีกรอบ **(ผลลัพธ์ควรจะได้ 403 Forbidden)**
- **ทดสอบว่าไม่ใส่ Token เลยเข้าได้ไหม:**
  ขึ้นไปกดปุ่ม Authorize แล้วกด Logout ออกให้หมด -> ลองยิง `GET /admin-only` **(ผลลัพธ์ควรจะได้ 401 Unauthorized)**

ลองเล่นดูครับ! ถ้าติด Error ตรงไหนก็อปปี้มาถามผมได้เลยครับ

code ในส่วนของ Authorize (มีรูปกุญแจ) คือตรงไหน

ปุ่ม **Authorize (รูปกุญแจ)** ในหน้าเว็บ Swagger นั้น ไม่ได้มาจากการเขียนโค้ดในไฟล์ JavaScript (`index.js`) โดยตรงครับ แต่เกิดจากการที่เราไปตั้งค่าไว้ในไฟล์ `swagger.yaml` ครับ

โค้ดที่สร้างรูปกุญแจและระบบใส่ Token แบ่งออกเป็น 2 ส่วนในไฟล์ `swagger.yaml` ดังนี้ครับ:

ส่วนที่ 1: โค้ดที่สร้างปุ่ม "Authorize" (ปุ่มใหญ่ด้านบนสุด)

อยู่ในหัวข้อ `components` -> `securitySchemes` ครับ โค้ดส่วนนี้เป็นการบอก Swagger ว่า "API ของเรามีการป้องกันนะ โดยใช้รูปแบบ HTTP Bearer (ส่ง Token)"

YAML

```yaml
components:
  securitySchemes:
    bearerAuth: # ชื่ออ้างอิง (ตั้งเป็นอะไรก็ได้ แต่ตั้งให้สื่อความหมาย)
      type: http
      scheme: bearer # ระบุว่าใช้รูปแบบ Bearer <token>
      bearerFormat: JWT # บอกว่าเป็น Token แบบ JWT
```

ส่วนที่ 2: โค้ดที่เอา "รูปแม่กุญแจ" ไปแปะที่แต่ละ Route (เช่น `/admin-only`)

สังเกตว่าที่ route `/login` จะไม่มีรูปกุญแจ แต่ที่ `/admin-only` จะมีรูปกุญแจ เพราะเราเอาคำว่า `security` ไปใส่ไว้ใน path นั้นครับ เพื่อบังคับว่าถ้าจะเรียกใช้ path นี้ ต้องแนบ Token (ที่ได้จากปุ่ม Authorize ด้านบน) มาด้วย

YAML

```yaml
  /admin-only:
    get:
      summary: เข้าถึงหน้าแดชบอร์ดของ Admin
      security:            # <--- ตรงนี้แหละครับที่ทำให้เกิดรูปกุญแจล็อคที่ Route นี้
        - bearerAuth: []   # อ้างอิงชื่อ bearerAuth มาจากส่วนที่ 1 ด้านบน
```

สรุปคือ:

- ฝั่ง **JavaScript (`index.js`)** ทำหน้าที่รับ Token มาเช็ค (ผ่าน Middleware ที่ชื่อ `authenticateToken` ที่เขียนว่า `req.headers['authorization']`)
- ฝั่ง `swagger.yaml` ทำหน้าที่สร้างปุ่ม (UI) ให้เรากรอก Token แล้ว Swagger จะเอา Token นั้นไปใส่ใน Header แบบ `Authorization: Bearer <token>` ให้อัตโนมัติเวลาเรากด Execute ครับ